Discrete Dynamic System with Hash Aggregation on Cyclic Slots

Zhandos Mambetali*
zhandos.mambetali@gmail.com

April 5, 2026

Abstract

We consider a discrete dynamic system of elements moving on a cyclic slot topology. Each element is characterized by a weight, speed, activation period, and initial position. For each slot, an aggregation function over the multiset of element values is introduced. A special case is cryptographic hash aggregation. The dynamics, periodicity, and informational properties of the system are formalized.

1 Introduction

Let $C = \{e_1, \ldots, e_n\}$ be a set of elements moving on slots

$$S = \mathbb{Z}_N = \{0, 1, \ldots, N-1\}$$

over discrete time $t \in \mathbb{Z}_{\geq 0}$.

Each element is defined as:

$$e = (w_e, v_e, k_e, t_0^e, p_0^e, f_e)$$

where:

- $w_e > 0$ — weight,
- $v_e \in \mathbb{Z}$ — speed,
- $k_e \in \mathbb{Z}_{>0}$ — update period,
- t_0^e — start time,
- $p_0^e \in S$ — initial position,
- $f_e \in \{0, 1\}$ — frozen flag.

MSC 2020: 37B15, 68Q80, 94A60. ORCID: 0000-0002-6767-8217

2 Related Work

The proposed system is related to several established models in discrete dynamics, distributed systems, and theoretical computer science.

Rotor-router models. Rotor-router systems describe deterministic analogues of random walks on graphs, where tokens move according to fixed cyclic rules. Similar to rotor-router dynamics, elements in our model follow deterministic motion on a discrete structure. However, in contrast to rotor-router models, where movement depends on local state (rotors), the proposed system introduces independent element velocities, activation periods, and global aggregation functions. Moreover, aggregation is not limited to counting or visitation frequency but supports arbitrary functions, including cryptographic hashes.

Consistent hashing. Consistent hashing is widely used in distributed systems to assign keys to nodes in a way that minimizes reallocation under system changes. The cyclic slot structure $S = \mathbb{Z}_N$ in our model resembles a hash ring. However, unlike classical consistent hashing, where key placement is static (or changes only upon rehashing), the proposed system introduces time-dependent dynamics of elements. Keys (elements) move continuously over the ring, and aggregation is performed at each time step, yielding a dynamic hashing process rather than a static mapping.

Cellular automata. Cellular automata (CA) define discrete-time evolution of grid-based systems with local interaction rules. The aggregation function F_s in our model plays a role analogous to a local update rule. However, classical CA operate on fixed cells with state transitions depending on neighboring cells. In contrast, our system separates motion and aggregation: elements move across slots independently, and aggregation is computed over the set of elements currently occupying each slot. This decoupling enables modeling of mobile agents and dynamic interaction patterns not captured by standard CA.

Multi-agent systems. The system can also be interpreted as a multi-agent system where agents move in a discrete environment and interact through shared resources (slots). Unlike typical multi-agent models that rely on explicit interaction rules between agents, interaction in the proposed system is implicit and mediated through aggregation functions, including nonlinear and non-invertible operators such as hashes.

Streaming and sketching algorithms. Aggregation over multisets with possible information loss relates to data stream processing and sketching techniques. However, classical sketches assume a static or sequential data stream, while the proposed system introduces a structured spatiotemporal evolution of data, where elements revisit slots periodically and aggregation evolves over time.

3 Novelty and Contributions

The novelty of the proposed model lies in the unified integration of deterministic motion, periodic activation, and generalized aggregation over a cyclic topology.

The main contributions are as follows:

- **Dynamic aggregation on cyclic structures.** We introduce a model where elements move on a cyclic group $\mathbb{Z}_N$ with individual velocities and activation periods, while slot-wise aggregation is performed at each time step. This defines a time-dependent aggregation process, in contrast to static or event-driven models.
- **Decoupling of motion and aggregation.** Unlike cellular automata and related systems, motion of elements is independent of aggregation, allowing flexible modeling of mobile agents and enabling richer dynamics.
- **Hash-based aggregation as a dynamical process.** We formalize the use of cryptographic hash functions as aggregation operators over evolving multisets, yielding a time-distributed hashing mechanism. This connects discrete dynamical systems with cryptographic primitives.
- **Periodicity analysis.** We derive explicit expressions for the period of individual elements and the overall system, showing that dynamics are governed by arithmetic properties of velocities and the cyclic domain.
- **Restricted dynamics with conflict resolution.** We extend the model to include slot restrictions and collision constraints between element classes, introducing competition, prioritization, and resource allocation mechanisms.
- **Unifying framework.** The model generalizes and connects several domains, including rotor-router systems, consistent hashing, cellular automata, and multi-agent dynamics, within a single formal structure.

In contrast to existing models, the proposed system combines mobility, aggregation, and nonlinearity (via hash functions) in a single framework, enabling analysis of dynamic distributed processes with both deterministic and information-theoretic properties.

4 Element Dynamics

$$p_e(t) = \begin{cases} p_0^e, & f_e = 1 \text{ or } t < t_0^e, \\ \left(p_0^e + v_e \cdot \lfloor \frac{t - t_0^e}{k_e} \rfloor\right) \bmod N, & \text{otherwise.} \end{cases} \tag{1}$$

This defines discrete dynamics on the group $\mathbb{Z}_N$.

5 Slot Aggregation

Let $\phi_e(t)$ denote the value of element e.

For slot s:

$$M_s(t) = \{\{\phi_e(t) \mid p_e(t) = s\}\}$$

is the multiset of values.

A canonical encoding is introduced:

$$\text{Encode}(M_s(t))$$

(e.g., sorting + serialization).

Then:

$$Y_s(t) = F_s(\text{Encode}(M_s(t)))$$

6 Hash Aggregation

If

$$F_s = \text{hash}_s$$

then:

$$Y_s(t) = \text{hash}_s(\text{Encode}(M_s(t)))$$

This defines a **time-distributed hash with moving keys**.

7 Periodicity

Theorem 1. *The period of an element is:*

$$T_e = k_e \cdot \frac{N}{\gcd(v_e, N)}$$

Proof. The motion is defined by linear dynamics modulo N. The period equals the order of v_e in $\mathbb{Z}_N$ taking discretization by k_e into account. □

The overall period:

$$T = \text{lcm}(T_1, \ldots, T_n)$$

8 Informational Properties

8.1 Information Loss

Let C be the system state.

Then:

$$H(C \mid Y(t)) > 0$$

i.e., aggregation leads to information loss.

8.2 Cryptographic Case

If cryptographic hashes are used:

- recovering C from Y is computationally hard,
- collisions are possible: different states yield the same $Y(t)$.

9 2D Generalization

Grid:

$$S = \mathbb{Z}_N \times \mathbb{Z}_M$$

Dynamics:

$$\begin{aligned} x_e(t) &= (x_0^e + v_e^x \cdot \lfloor \frac{t - t_0^e}{k_e} \rfloor) \bmod N \\ y_e(t) &= (y_0^e + v_e^y \cdot \lfloor \frac{t - t_0^e}{k_e} \rfloor) \bmod M \end{aligned} \tag{2}$$

Aggregation:

$$Y_{x,y}(t) = F_{x,y}(\text{Encode}(M_{x,y}(t)))$$

where:

$$M_{x,y}(t) = \{\{\phi_e(t) \mid (x_e, y_e) = (x, y)\}\}$$

10 Interpretation

The system is equivalent to:

- a cellular automaton with external agents,
- a dynamic hash system,
- a particle model on a torus.

11 Restricted Subsets and Collision Prohibition

11.1 Partitioning Elements into Sets

Let elements be partitioned into subsets:

$$C = \bigsqcup_{\alpha \in A} C_\alpha$$

where A is the set of labels (types), and C_α are elements of type α.

Each type α has a subset of allowed slots:

$$S_\alpha \subseteq S$$

11.2 Restricted Dynamics

For element $e \in C_\alpha$, its position is restricted:

$$p_e(t) \in S_\alpha \quad \forall t$$

Movement follows the cyclic order within S_α.

Let $S_\alpha = \{s_0, s_1, \ldots, s_{m-1}\}$ be an ordered list. Define local index:

$$i_e(t) \in \mathbb{Z}_m$$

Dynamics:

$$i_e(t) = \begin{cases} i_0^e, & t < t_0^e \text{ or } f_e = 1, \\ i_0^e + v_e \cdot \left\lfloor \frac{t - t_0^e}{k_e} \right\rfloor \mod m, & \text{otherwise}, \end{cases}$$

$$p_e(t) = S_\alpha[i_e(t)]$$

11.3 Global Shift

A global shift can be introduced:

$$p'_e(t) = (p_e(t) + g(t)) \mod N$$

where $g(t)$ is a shift function (e.g., $g(t) = t$).

11.4 Collision and Overlap Prohibition

Let elements in slot s be:

$$E_s(t) = \{e \mid p_e(t) = s\}$$

Constraint:

$$\forall e_i, e_j \in E_s(t): \quad \alpha_i \neq \alpha_j \Rightarrow \text{prohibited}$$

Elements from different sets cannot occupy the same slot.

11.5 Conflict Resolution

Define a conflict resolution operator:

$$\mathcal{R}(E_s(t)) \subseteq E_s(t)$$

Strategies:

- discard conflicting elements,
- priority by weight,
- random selection,
- keep only one set.

Then:

$$\tilde{E}_s(t) = \mathcal{R}(E_s(t))$$

and aggregation:

$$Y_s(t) = F_s(\text{Encode}(\{\{\phi_e(t) \mid e \in \tilde{E}_s(t)\}\}))$$

11.6 Properties

- the system becomes a **restricted dynamic system**,
- competition for resources (slots) arises,
- dynamics depend on conflict resolution policy,
- potential deadlocks or set domination may occur.

12 Illustrative Example: Eratosthenes-like Dynamics

We demonstrate how the proposed system can emulate behavior analogous to the classical Sieve of Eratosthenes through deterministic motion on cyclic slots.

12.1 Setup

Let $S = \mathbb{Z}_N$ be the set of slots. Consider a collection of elements

$$C = \{e_1, e_2, \dots, e_k\}$$

where each element e_i is associated with an integer step size $v_i > 1$.

Each element moves according to:

$$p_i(t+1) = (p_i(t) + v_i) \bmod N$$

We interpret each element e_i as generating a periodic traversal over slots corresponding to multiples of v_i.

12.2 Coverage Interpretation

Define the visitation indicator:

$$\chi_s(t) = \begin{cases} 1, & \text{if there exists } e_i \text{ such that } p_i(t) = s \\ 0, & \text{otherwise} \end{cases}$$

We then define cumulative coverage over time:

$$\mathcal{X}_s(T) = \sum_{t=0}^{T} \chi_s(t)$$

Slots s for which $\chi_s(T) = 0$ (excluding initial conditions) correspond to positions not visited by any element trajectory.

12.3 Analogy to the Sieve of Eratosthenes

If elements are initialized with step sizes corresponding to prime numbers:

$$v_i \in \{2, 3, 5, 7, \dots\}$$

then each element marks positions corresponding to multiples of its step size. Over time, the system covers all composite indices, while certain slots remain unvisited.

These unvisited slots approximate prime positions within the cyclic domain, up to boundary and periodicity effects induced by modulo arithmetic.

12.4 Dynamical Perspective

Unlike the classical sieve, which operates by sequential elimination, the proposed system:

- performs marking *in parallel* via independent moving elements,
- distributes the computation over time rather than performing explicit elimination,
- introduces periodic revisitation due to cyclic structure,
- allows extension with stochasticity, priorities, and constraints.

12.5 Interpretation

This construction demonstrates that the proposed framework can encode number-theoretic filtering processes as spatiotemporal dynamics.

In particular:

- composite numbers correspond to slots with high visitation frequency,
- prime-like positions correspond to low-frequency or zero-frequency slots,
- the system acts as a distributed, time-evolving sieve.

12.6 Extensions

The example can be extended by:

- introducing weighted elements to bias marking intensity,
- restricting allowed slots S_α to simulate segmented sieves,
- applying nonlinear aggregation (e.g., hash functions) to study information mixing over arithmetic structures.

13 Examples and Applications

13.1 Distributed Systems

Interpret elements e as tasks or data keys, slots s as compute nodes, and sets C_α as task classes (e.g., services or priorities).

Allowed slots $S_\alpha \subseteq S$ define node affinity or shard placement.

Dynamics $p_e(t)$ describe **task migration**:

$$p_e(t+1) = (p_e(t) + v_e) \mod N$$

Global shift $g(t)$ models:

- load redistribution,
- rebalancing,
- hash-ring changes.

Conflicts model:

- node resource limits,
- isolation policies (multi-tenant systems),
- forbidden co-location.

Thus, the system is a **deterministic dynamic sharding model**.

13.2 Scheduling Theory

Elements are tasks, slots are time intervals or processors, k_e is task period, t_0^e is readiness time.

$$p_e(t)$$

specifies when/where a task executes.

Slot restrictions S_α define allowable processors, time windows, or compatibility constraints.

Overlap prohibition:

$$\forall e_i \neq e_j : \; p_{e_i}(t) = p_{e_j}(t) \Rightarrow \text{conflict}$$

corresponds to **collision-free scheduling**.

Conflict operator $\mathcal{R}$ defines scheduler strategy:

- priority (by w_e),
- round-robin,
- random,
- greedy.

Hence, the system is a **dynamic periodic-task scheduling model**.

13.3 Complex Dynamic Systems

System as a whole:

$$Y(t) = \mathcal{F}(P(t), X(t))$$

exhibits properties of complex systems:

- **self-organization**: slot distribution forms without centralized control,
- **nonlinearity**: with nonlinear F_s (e.g., hashes),
- **parameter sensitivity**: small changes in k_e, v_e, or S_α yield qualitatively different dynamics,
- **emergence**: global patterns arise from local rules.

In particular:

- linear $F_s \to$ **kinetic particle model**,
- hash aggregation $\to$ **entropy mixer**,
- with restrictions $\to$ **multi-agent interaction system**.

13.4 Relation to Known Models

The system generalizes:

- rotor-router models (deterministic motion),
- consistent hashing (key distribution),
- cellular automata (local state aggregation),
- multi-agent systems (agent interactions).

Key feature:

element motion and aggregation occur simultaneously over time.

14 Kinematic Analogy and Inertial Frames

Each element e is a discrete particle with mass $m_e = w_e$, position

$$\vec{r}_e(t) = (x_e(t), y_e(t))$$

and velocity

$$\vec{v}_e = (v_e^x, v_e^y)$$

Dynamics:

$$\vec{r}_e(t+1) = \vec{r}_e(t) + \vec{v}_e \mod (N, M)$$

14.1 Inertial Reference Frames

Consider a frame moving with velocity $\vec{u}$:

$$\vec{r}_e'(t) = \vec{r}_e(t) - \vec{u}t$$
$$\vec{v}_e' = \vec{v}_e - \vec{u}$$

The system is invariant under discrete analogs of Galilean transformations.

14.2 Velocity Addition and Local Momentum

For a cell (x, y) with elements, linear aggregation:

$$F_{x,y} = \sum$$

yields:

$$Y_{x,y}(t) = \sum_{e:\,(x_e,y_e)=(x,y)} m_e \vec{v}_e$$

Equivalent to **total momentum** in the cell.
Average velocity (local frame):

$$\vec{u}_{x,y}(t) = \frac{\sum m_e \vec{v}_e}{\sum m_e}$$

Switch to local frame:

$$\vec{v}_e' = \vec{v}_e - \vec{u}_{x,y}(t)$$
$$\sum m_e \vec{v}_e' = 0$$

corresponding to the center-of-mass frame.

14.3 Interpretation

- each cell (x, y) defines a **local inertial frame**,
- aggregated $Y_{x,y}(t)$ corresponds to momentum,
- normalized system $(\vec{u}_{x,y}) \to$ center-of-mass frame,
- model dynamics allow reference frame changes without altering motion structure.

14.4 Nonlinear Case

If $F_{x,y}$ is nonlinear (e.g., hash):

- total momentum information is lost,
- invariance under frame change breaks,
- system loses physical interpretation but gains cryptographic properties.

15 Applications

- streaming / sketching,
- distributed systems,
- pseudorandom generators,
- time-based hashing,
- self-organization models.

15.1 On Turing Completeness

In the current formalization, the system is not Turing-complete.

With a fixed number of elements C and slots S, the system state space is finite. Thus, dynamics $P(t)$ are periodic, and the system lacks unbounded memory required for universal computation.

However, the model can be extended to become Turing-complete by introducing one or more of:

- unbounded number of slots (expandable "tape"),
- mechanisms for changing internal element states $\phi_e(t)$,
- local interaction rules between neighboring slots.

With these extensions, the system can simulate known Turing-complete computation models (e.g., cellular automata) and becomes Turing-complete.

Thus, the proposed construction serves as a base dynamic model with natural extension to a universal computing system.

16 Conclusion

We propose a universal model of dynamic aggregation with moving elements. The system combines properties of dynamic systems, cellular automata, and cryptographic hashes, making it suitable for analyzing complex distributed processes.

References

[1] L. Levine, *The Rotor-Router Model*, arXiv:math/0409407, 2004.

[2] D. A. Levin, Y. Peres, *Markov Chains and Mixing Times*, American Mathematical Society, 2009.

[3] A. E. Holroyd et al., *Chip-Firing and Rotor-Routing on Directed Graphs*, In and Out of Equilibrium 2, Birkhäuser, 2008.

[4] L. Levine, Y. Peres, *Strong spherical asymptotics for rotor-router aggregation and the divisible sandpile*, Potential Analysis, 30(1), 1–27, 2009.

[5] D. Karger et al., *Consistent Hashing and Random Trees: Distributed Caching Protocols for Relieving Hot Spots on the World Wide Web*, Proc. ACM STOC, 1997.

[6] M. Mitzenmacher, *The Power of Two Choices in Randomized Load Balancing*, IEEE Transactions on Parallel and Distributed Systems, 2001.

[7] S. Wolfram, *A New Kind of Science*, Wolfram Media, 2002.

Our dataset is published on Zenodo: DOI: 10.5281/zenodo.19486289.